

Reporte de Evaluación Técnica

Información General

Candidato / Identificador:	Candidato E2E - 2026-03-12T03:08:29.924Z
Fecha y hora de evaluación:	11/03/2026 22:08

Nota Global

3/5 - Aceptable

Rúbrica Utilizada

#	Criterio	Descripción
1	Legibilidad del código	Claridad, nombres descriptivos, estructura
2	Buenas prácticas	Patrones, convenciones, organización
3	Eficiencia	Complejidad algorítmica, uso de recursos
4	Manejo de errores	Validaciones, excepciones, casos límite
5	Cumplimiento del enunciado	Requisitos funcionales cumplidos

Evaluación por Criterio

Criterio	Nota	Comentarios
Legibilidad del código	4/5	El código es claro y conciso. La función tiene un nombre descriptivo ' suma' que indica claramente su propósito. Los parámetros 'a' y 'b' son simples pero apropiados para una función básica. La estructura es limpia con una sola línea de retorno.
Buenas prácticas	2/5	El código carece de documentación (docstring), type hints y validaciones. No sigue las convenciones de PEP 8 para documentación de funciones. La función es demasiado genérica sin especificar el tipo de datos esperados o el comportamiento con diferentes tipos.
Eficiencia	5/5	La implementación es óptima con complejidad O(1) en tiempo y espacio. Utiliza el operador nativo '+' de Python que es la forma más eficiente de realizar la suma. No hay operaciones redundantes ni uso innecesario de memoria.

Criterio	Nota	Comentarios
Manejo de errores	1/5	No existe manejo de errores. La función no valida tipos de entrada, no maneja casos donde los operandos no son sumables, y no proporciona mensajes de error informativos. Fallará silenciosamente o con errores poco claros en casos límite.
Cumplimiento del enunciado	3/5	Sin un enunciado específico, se asume que debe realizar una suma básica, lo cual cumple. Sin embargo, para ser una solución completa, necesitaría más robustez y especificaciones claras sobre el comportamiento esperado.

Resumen Ejecutivo

El código presenta una implementación básica y funcionalmente correcta de una función suma. Destaca por su eficiencia y legibilidad, pero carece significativamente de robustez, documentación y manejo de errores. Es apropiado para casos de uso simples pero inadecuado para entornos de producción.

Fortalezas

- Implementación eficiente con complejidad $O(1)$
- Código limpio y fácil de entender
- Funcionalidad básica correcta
- Uso apropiado del operador nativo de suma

Áreas de Mejora

- Agregar documentación (docstring) y type hints
- Implementar validación de tipos de entrada
- Incluir manejo de excepciones y casos límite
- Considerar casos edge como suma de diferentes tipos de datos

Reporte de Evaluación de Hoja de Vida (CV)

Información General

Candidato / Identificador:	Candidato E2E - 2026-03-12T03:08:29.924Z
Fecha y hora de evaluación:	11/03/2026 22:09
Origen:	text

Match Score

4/5 - Alto

Resumen Ejecutivo

Candidato sólido con experiencia directa en los requisitos principales del rol. Cuenta con 3 años desarrollando APIs con Python y FastAPI, manejo de Git, pruebas automatizadas y tecnologías complementarias como Docker y CI/CD. Su perfil técnico se alinea bien con las necesidades del puesto, aunque el CV carece de detalles sobre proyectos específicos y experiencia en trabajo en equipo.

Fortalezas

- 3 años de experiencia específica desarrollando APIs con Python
- Conocimiento de FastAPI para desarrollo de APIs REST
- Experiencia en pruebas automatizadas
- Manejo de Git como requerido
- Conocimientos en Docker y CI/CD que agregan valor
- Experiencia en SQL para manejo de bases de datos
- Perfil técnico alineado con stack tecnológico moderno

Brechas (Gaps)

- No se evidencia experiencia específica en trabajo en equipo
- No se detallan ejemplos de manejo de errores implementados
- Falta información sobre metodologías de desarrollo o buenas prácticas aplicadas
- No se mencionan proyectos específicos o logros cuantificables
- Ausencia de certificaciones o formación académica
- No se evidencia experiencia con frameworks de testing específicos

Recomendaciones

- Agregar ejemplos específicos de proyectos de APIs desarrolladas con métricas de impacto
- Incluir detalles sobre experiencia trabajando en equipos ágiles o colaborativos
- Mencionar estrategias específicas de manejo de errores implementadas
- Detallar frameworks de testing utilizados (pytest, unittest, etc.)

- Agregar información sobre formación académica o certificaciones relevantes
- Incluir ejemplos de buenas prácticas de desarrollo aplicadas
- Especificar metodologías de desarrollo conocidas (Scrum, Kanban, etc.)
- Cuantificar logros y resultados obtenidos en proyectos anteriores

Palabras clave alineadas

Python, APIs REST, Git, pruebas automatizadas, FastAPI, Docker, CI/CD, SQL, backend, desarrollo, 3 años experiencia